DAA LAB — WEEK 5
Programs 20–25
Source: extracted from your uploaded DAA lab PDF.

==
PROGRAM 20: Find duplicate in array (Floyd cycle detection)
==

```
#include <stdio.h>
int main() {
   int n;
   scanf("%d", &n);
   int arr[n];
   for (int i = 0; i < n; i++)

       scanf("%d", &arr[i]);
   int slow = arr[0];
   int fast = arr[0];
   do {
       slow = arr[slow];
       fast = arr[arr[fast]];
   } while (slow != fast);
   slow = arr[0];
   while (slow != fast) {
       slow = arr[slow];
       fast = arr[fast];
   }
   printf("%d", slow);
   return 0;
}
```

==
PROGRAM 21: Find duplicate in array (XOR approach)
==

```
#include <stdio.h>
int main() {
   int n;
   scanf("%d", &n);
   int arr[n];
   int result = 0;
   for (int i = 0; i < n; i++) {
       scanf("%d", &arr[i]);

       result ^= arr[i];
   }
   for (int i = 1; i <= n - 1; i++)
       result ^= i;
   printf("%d", result);
   return 0;
}
```

```
================================================================================
PROGRAM 22: Intersection of two sorted arrays (two pointers)
================================================================================

#include <stdio.h>
int main() {
   int T;
   scanf("%d", &T);
   while (T--) {
       int n1, n2;
       scanf("%d", &n1);
       int a[n1];
       for (int i = 0; i < n1; i++)
           scanf("%d", &a[i]);
       scanf("%d", &n2);
       int b[n2];
       for (int i = 0; i < n2; i++)
           scanf("%d", &b[i]);
       int i = 0, j = 0;
       int first = 1;
       while (i < n1 && j < n2) {
           if (a[i] == b[j]) {
               if (!first)
                   printf(" ");
               printf("%d", a[i]);
               first = 0;
               i++;
               j++;
           }
           else if (a[i] < b[j]) {
               i++;
           }
           else {
               j++;
           }
       }
       printf("\n");

   }
   return 0;
}


================================================================================
PROGRAM 23: Intersection of two sorted arrays (two pointers, duplicate
version)
================================================================================

#include <stdio.h>
int main() {
   int T;
   scanf("%d", &T);
   while (T--) {
```

```
        int n1, n2;
        scanf("%d", &n1);
        int a[n1];
        for (int i = 0; i < n1; i++)
            scanf("%d", &a[i]);
        scanf("%d", &n2);
        int b[n2];
        for (int i = 0; i < n2; i++)
            scanf("%d", &b[i]);
        int i = 0, j = 0;
        int first = 1;
        while (i < n1 && j < n2) {
            if (a[i] == b[j]) {
                if (!first)
                    printf(" ");
                printf("%d", a[i]);
                first = 0;
                i++;
                j++;
            } else if (a[i] < b[j]) {
                i++;
            } else {
                j++;
            }
        }
        printf("\n");
    }
    return 0;
}
```

==========

PROGRAM 24: Find pair with difference k (two pointers)

==========

```
#include <stdio.h>
int main() {
    int n, k;
    scanf("%d", &n);
    int a[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    scanf("%d", &k);
    int i = 0, j = 1;
    while (i < n && j < n) {
        if (i == j) {
            j++;
        } else {
            int diff = a[j] - a[i];
            if (diff == k) {

                printf("1");
                return 0;
            } else if (diff < k) {
```

```
            j++;
        } else {
            i++;
        }
    }
  }
  printf("0");
  return 0;
}
```

==========
PROGRAM 25: Find pair with difference k (two pointers, duplicate version)
==========

```
#include <stdio.h>
int main() {
  int n, k;
  scanf("%d", &n);

  int a[n];
  for (int i = 0; i < n; i++)
      scanf("%d", &a[i]);
  scanf("%d", &k);
  int i = 0, j = 1;
  while (i < n && j < n) {
      if (i == j) {
          j++;
          continue;
      }
      if (a[j] - a[i] == k) {
          printf("1");
          return 0;
      }
      if (a[j] - a[i] < k)
          j++;
      else
          i++;
  }
  printf("0");
  return 0;
}
```